

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT

on

Artificial Intelligence (23CS5PCAIN)

Submitted by

Chaithanya Sudhan (1BM23CS073)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)



BENGALURU-560019

Sep-2025 to Jan-2026

B.M.S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the Lab work entitled “Artificial Intelligence (23CS5PCAIN)” carried out by **Chaithanya Sudhan (1BM23CS073)**, who is bonafide student of **B.M.S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum. The Lab report has been approved as it satisfies the academic requirements in respect of an Artificial Intelligence (23CS5PCAIN) work prescribed for the said degree.

Swathi Sridharan Assistant Professor Department of CSE, BMSCE	Dr. Kavitha Sooda Professor & HOD Department of CSE, BMSCE
---	--

Index

Sl. No.	Date	Experiment Title	Page No.
1	20-8-2025	Implement Tic –Tac –Toe Game Implement vacuum cleaner agent	1 - 5
2	28-8-2025	Implement 8 puzzle problems using Depth First Search (DFS) Implement Iterative deepening search algorithm	6 - 9
3	3-9-2025	Implement A* search algorithm	10 - 13
4	10-9-2025	Implement Hill Climbing search algorithm to solve N-Queens problem	14 - 16
5	17-9-2025	Simulated Annealing to Solve 8-Queens problem	17 - 19
6	24-9-2025	Create a knowledge base using propositional logic and show that the given query entails the knowledge base or not.	20 - 21
7	8-10-2025	Implement unification in first order logic	22 - 24
8	15-10-2025	Create a knowledge base consisting of first order logic statements and prove the given query using forward reasoning.	25 - 29
9	29-10-2025	Create a knowledge base consisting of first order logic statements and prove the given query using Resolution	29 - 32
10	12-11-2025	Implement Alpha-Beta Pruning.	33 - 35

<https://github.com/chaithanyasudhan/AI>

Implement vacuum cleaner agent

2/12/22

Algorithms

1. Algorithm

2. start

3. Initialize 3x3 matrix

4. Ask the user if they want to start first

5. loop with matrix range if user starts they place at any position if system starts it puts at any random position

6. assign winning positions = { (0,1,2) (3,4,5) (6,7,8) (0,3,6) (1,4,7) (2,5,8) (2,4,6) }

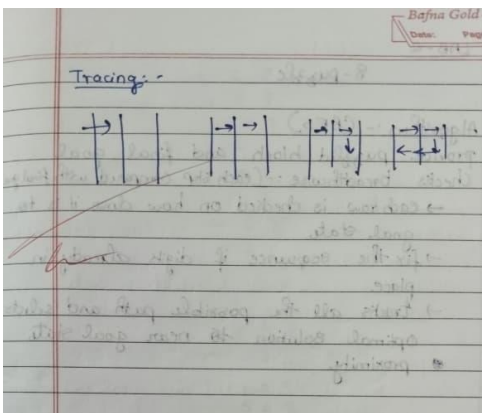
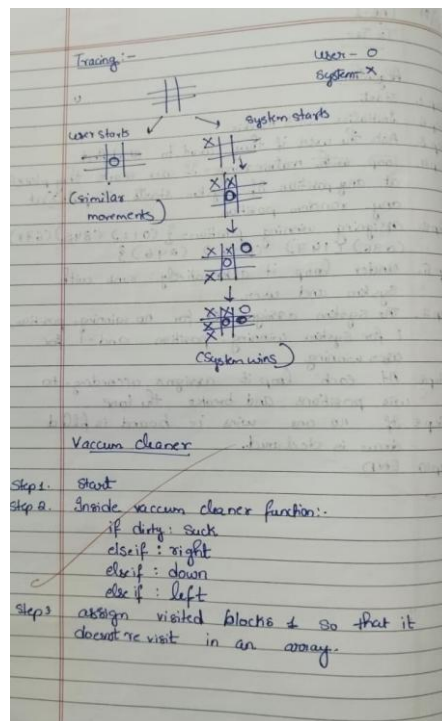
7. under loop if alternatively runs with system and user

8. The system assigns 0 for no winning position, 1 for system winning position and -1 for user winning

9. At each loop it assigns according to win positions and breaks the loop

10. If no one wins i.e. board is filled draw is declared.

11. END



Code:

```
#Tic_Tac_Toe

def print_board(board):
    for row in board:
        print(" | ".join(row))
        print("--" * 5)

def check_winner(board, player):
    # Check rows
    for row in board:
        if all(cell == player for cell in row):
            return True

    # Check columns
    for col in range(3):
        if all(board[row][col] == player for row in range(3)):
            return True

    # Check diagonals
    if all(board[i][i] == player for i in range(3)) or \
        all(board[i][2 - i] == player for i in range(3)):
        return True

    return False

def tic_tac_toe():
    board = [["_ " for _ in range(3)] for _ in range(3)]
    players = ["X", "O"]
    moves = 0

    while moves < 9:
        print_board(board)
        player = players[moves % 2]
        print(f"Player {player}'s turn")

        row = int(input("Enter row (0-2): "))
        col = int(input("Enter col (0-2): "))
```

```

        if board[row][col] == " ":
board[row][col] = player
moves += 1      else:
        print("Cell already taken, try again!")
continue
        if check_winner(board, player):
print_board(board)      print(f"Player
{player} wins!")
        return

print_board(board)
print("It's a draw!")

# Run the game tic_tac_toe()

#VacuumCleaner

import random

# Environment: 2 rooms A and B, both dirty initially environment
= {
    "A": "Dirty",
    "B": "Dirty"
}

# Simple Reflex Agent actions def
simple_reflex_agent(location, environment):
    actions_taken = []
    current_location = location

    for _ in range(2): # Maximum two locations: A and B
status = environment[current_location]      if status ==
"Dirty":
        actions_taken.append(f"Clean {current_location}")
environment[current_location] = "Clean"      else:
        actions_taken.append(f"No cleaning needed at {current_location}")

    # Move to the other room      if
current_location == "A":
actions_taken.append("Move Right to B")
        current_location = "B"
else:
        actions_taken.append("Move Left to A")
        current_location = "A"

```

```

    return actions_taken, environment

# Goal-Based Agent simulation with step-by-step actions def
goal_based_agent_simulation(environment, start_location):
    actions = []    env = environment.copy()
    current_location = start_location

    # Check current room    if
    env[current_location] == "Dirty":
        actions.append(f"Clean {current_location}")
    env[current_location] = "Clean"    else:
        actions.append(f"No cleaning needed at {current_location}")

    # Move to other room
    other_location = "B" if current_location == "A" else "A"
    actions.append(f"Move to {other_location}")    current_location
    = other_location

    # Check other room    if env[current_location]
    == "Dirty":        actions.append(f"Clean
    {current_location}")        env[current_location]
    = "Clean"    else:
        actions.append(f"No cleaning needed at {current_location}")

    return actions, env

# Run simulation printing all steps def
run_simulation():
    print("Initial Environment:", environment)

    # Start location randomly chosen    start_location = random.choice(["A", "B"])
    print(f"\nSimple Reflex Agent starts at {start_location}")    reflex_actions, reflex_env
    = simple_reflex_agent(start_location, environment.copy())    for step, action in
    enumerate(reflex_actions, 1):
        print(f"Step {step}: {action}")
    print("Environment after Reflex Agent:", reflex_env)

    print(f"\nGoal-Based Agent starts at {start_location}")    goal_actions, goal_env =
    goal_based_agent_simulation(environment, start_location)    for step, action in
    enumerate(goal_actions, 1):
        print(f"Step {step}: {action}")    print("Environment
    after Goal-Based Agent:", goal_env)

```


Output:

#Tic_Tac_Toe

```

===== RES:ARI: C:\ALIAS\TIO_TIO_
| |
| |
| |
Player X's turn
Enter row (0-2): 1
Enter col (0-2): 1
| |
| X |
| |
| |
=====
Player O's turn
Enter row (0-2): 2
Enter col (0-2): 1
| |
| X |
| O |
| |
Player X's turn
Enter row (0-2): 0
Enter col (0-2): 0
X | |
| X |
| O |
| |
Player O's turn
Enter row (0-2): 2
Enter col (0-2): 1
| |
| X |
| O | X
| |
Player X wins!

Player O's turn
Enter row (0-2): 2
Enter col (0-2): 0
X | |
| X |
| O |
| |
Player O's turn
Enter row (0-2): 2
Enter col (0-2): 2
X | |
| X |
| O |
| |
Player X's turn
Enter row (0-2): 2
Enter col (0-2): 2
X | |
| X |
| O | X
| |
Player X wins!

```

#VaccumCleaner

```
Initial Environment: {'A': 'Dirty', 'B': 'Dirty'}

Simple Reflex Agent starts at B
Step 1: Clean B
Step 2: Move Left to A
Step 3: Clean A
Step 4: Move Right to B
Environment after Reflex Agent: {'A': 'Clean', 'B': 'Clean'}

Goal-Based Agent starts at B
Step 1: Clean B
Step 2: Move to A
Step 3: Clean A
Environment after Goal-Based Agent: {'A': 'Clean', 'B': 'Clean'}
```

Program 2

Implement 8 puzzle problems using Depth First Search (DFS)
Implement Iterative deepening search algorithm

Algorithm:

11/9/22

TODDS

Algorithm:-

- Step 1: Start
- Step 2: Initialise start node and Goal node
- Step 3: Iterate till max depth or goal node found:
 - start DFS (array [1])
 - from root node traverse to its child nodes using BFS & update in array.
 - Check if goal node reached in each iteration

Steps: If goal node reached print the path

Else if max depth reached and goal not reached no solution

Steps: End

Ex:

```
graph TD; A((A)) --- B((B)); A --- C((C)); B --- D((D)); B --- E((E)); C --- F((F)); C --- G((G)); D --- H((H)); E --- I((I)); style G stroke-width:4px
```

Start DFS(A)

- ↳ B, C (Using BFS)

DFS(B)

- ↳ D, E (Using BFS)

DFS(C)

- ↳ F, G (Using BFS)

A → B → D → E → C → F → G

(Goal reached)

for goal node X

~~Start~~ DFS(A)

- ↳ B, C (using BFS)
- ↳ DFS(B)
- ↳ D, E (using BFS)
- ↳ DFS(C)
- ↳ F, G (using BFS)
- ↳ DFS(D)
- ↳ H (no BFS)
- ↳ DFS(E)
- ↳ I (goal node reached)

A → B → D → H → E → ~~X~~ → C → F → G

Tracing for 8 puzzle

1 2 3	1 2 3
- 5 6	→ 4 5 6
4 7 8	- 7 8
	↓
1 2 3	1 2 3
4 5 6	← 4 5 6
7 8	7 - 8

(goal state reached)

graph path: D → R → R

using IDDFS

2/3/20

LAB-2

8-puzzle

Algorithm:- (BFS)

Step 1:- provide puzzle block and final goal

Step 2:- checks breadthwise:- (each step compared with final goal state)

- each row is checked on how close it is to goal state
- fix the sequence if digits already in place
- tracks all the possible path and selects optimal solution to near goal state

- proximity

Tracing

```
graph TD
    A["1 2 3  
4 5  
6 7 8"] --> B["1 2 3  
4 5 8  
6 7"]
    A --> C["1 2 3  
4 5 8  
6 7"]
    B --> D["1 2 3  
4 5 8  
6 7"]
    C --> E["1 2 3  
4 5 8  
6 7"]
```

Diagram illustrating the tracing of the 8-puzzle solution using Breadth-First Search (BFS). The initial state is a 3x3 grid with tiles 1, 2, 3 in the top row, 4, 5, 6 in the middle row, and 7, 8, and an empty space in the bottom row. The goal state is 1, 2, 3 in the top row, 4, 5, 8 in the middle row, and 6, 7, and an empty space in the bottom row. The diagram shows the search tree starting from the initial state, branching into two paths, and then further branching into more paths, illustrating the search process.

1 2 3
4 5 8
6 7
↓
1 2 3
5 8
4 4 7
↓
1 2 3
5 3
4 6 7
↖ ↗
1 2 3 4
5 6 8
4 7
↓
1 2 3
(4 5 6)
7 8
↓
1 2 3
4 5 6 ← goal state
7 8

Code:

```
from collections import deque import
copy
```

```
target_config = [[1, 2, 3],
                  [4, 5, 6],
                  [7, 8, 0]]
```

```
directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]
```

```
def locate_blank(board):
```

```
    for row in range(3):    for
col in range(3):            if
board[row][col] == 0:
        return row, col
```

```
def generate_next_states(board):
```

```
    next_states = []    row, col =
locate_blank(board)    for dr, dc
in directions:
        new_row, new_col = row + dr, col + dc
if 0 <= new_row < 3 and 0 <= new_col < 3:
        temp_board = copy.deepcopy(board)        temp_board[row][col],
temp_board[new_row][new_col] = temp_board[new_row][new_col], temp_board[row][col]
next_states.append(temp_board)    return next_states
```

```
def reached_goal(board):
```

```
    return board == target_config
```

```
def display_board(board):
```

```
    for row in board:
        print(row)
    print()
```

```
def depth_first_search(initial_board, max_depth=50):
```

```
    stack = [(initial_board, [])]
    explored = set()
```

```

while stack:
    current_board, moves_so_far = stack.pop()
    board_key = tuple(tuple(r) for r in current_board)

    if board_key in explored:
        continue
    explored.add(board_key)

    if reached_goal(current_board):
        return moves_so_far + [current_board]

    if len(moves_so_far) >= max_depth:
        continue

    for new_board in generate_next_states(current_board):
        stack.append((new_board, moves_so_far + [current_board]))
    return None

def depth_limited_search(board, depth, path_trace, seen):

    if reached_goal(board):
        return path_trace + [board]
    if depth == 0:
        return None

    board_signature = tuple(tuple(r) for r in board)
    seen.add(board_signature)

    for successor in generate_next_states(board):
        successor_key = tuple(tuple(r) for r in successor)
        if successor_key not in seen:
            outcome = depth_limited_search(successor, depth - 1, path_trace + [board], seen)
            if outcome:
                return outcome
    return None

def iterative_deepening_search(start_board, depth_limit=50):

    for depth in range(depth_limit):
        visited_nodes = set()
        result_path =
        depth_limited_search(start_board, depth, [], visited_nodes)
    result_path:

```

```

        return result_path
return None
if __name__ == "__main__":
    puzzle_start = [[1, 2, 3],
                    [4, 0, 6],
                    [7, 5, 8]]

    print("DFS Result:")    dfs_result =
depth_first_search(puzzle_start, max_depth=20)    if
dfs_result:        for state in dfs_result:
display_board(state)    else:
    print("No solution found using DFS.")

    print("\nIDS Result:")    ids_result =
iterative_deepening_search(puzzle_start, depth_limit=20)
    if ids_result:
    for state in ids_result:
display_board(state)
    else:
        print("No solution found using IDS.")

```

Output:

```

DFS Solution:
[1, 2, 3]
[4, 0, 6]
[7, 5, 8]

[1, 2, 3]
[4, 6, 0]
[7, 5, 8]

[1, 2, 3]
[4, 6, 8]
[7, 5, 0]

[1, 2, 3]
[4, 6, 8]
[7, 0, 5]

[1, 2, 3]
[4, 6, 8]
[0, 7, 5]

[1, 2, 3]
[0, 6, 8]
[4, 7, 5]

[1, 2, 3]
[6, 0, 8]
[4, 7, 5]

[1, 2, 3]
[6, 7, 8]
[4, 0, 5]

[1, 2, 3]
[6, 7, 8]
[0, 4, 5]

[1, 2, 3]
[0, 7, 8]
[6, 4, 5]

[1, 2, 3]
[7, 0, 8]
[6, 4, 5]

[1, 2, 3]
[7, 4, 8]
[6, 0, 5]

[1, 2, 3]
[7, 4, 8]
[0, 6, 5]

[1, 2, 3]
[0, 4, 8]
[7, 6, 5]

[1, 2, 3]
[4, 0, 8]
[7, 6, 5]

[1, 2, 3]
[4, 8, 0]
[7, 6, 5]

[1, 2, 3]
[4, 8, 5]
[7, 6, 0]

[1, 2, 3]
[4, 8, 5]
[7, 0, 6]

[1, 2, 3]
[4, 0, 5]
[7, 8, 6]

[1, 2, 3]
[4, 5, 0]
[7, 8, 6]

[1, 2, 3]
[4, 5, 6]
[7, 8, 0]

IDS Solution:
[1, 2, 3]
[4, 0, 6]
[7, 5, 8]

[1, 2, 3]
[4, 5, 6]
[7, 0, 8]

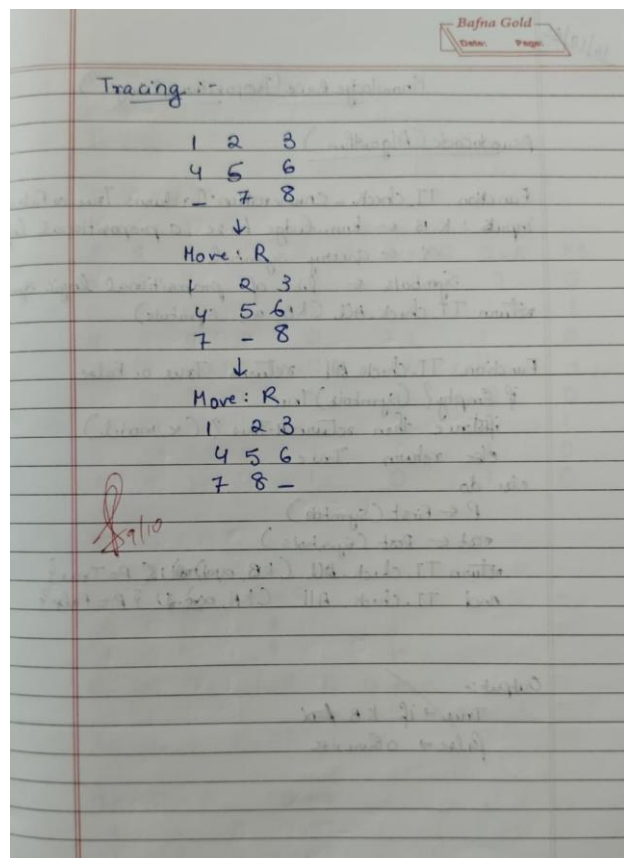
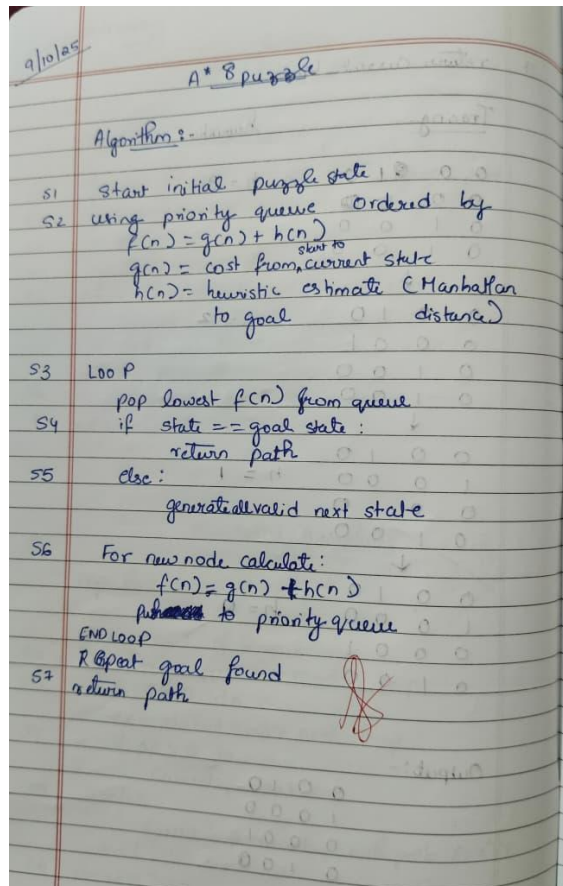
[1, 2, 3]
[4, 5, 6]
[7, 8, 0]

```

Program 3

Implement A* search algorithm

Algorithm:



Code:

```
import heapq
```

```
target = [[1,2,3],  
          [8,0,4],  
          [7,6,5]]
```

```
directions = [(1,0), (-1,0), (0,1), (0,-1)]
```

```
def board_to_tuple(board):
```

```

    return tuple(tuple(row) for row in board)

def locate(board, val):
    for r in range(3):
        for c in range(3):
            if board[r][c] == val:
                return (r, c)

def misplaced_tiles_heuristic(board):
    misplaced = 0
    for r in range(3):
        for c in range(3):
            if board[r][c] != 0 and board[r][c] != target[r][c]:
                misplaced += 1
    return misplaced

def manhattan_heuristic(board):
    total_dist = 0
    for r in range(3):
        for c in range(3):
            val = board[r][c]
            if val != 0:
                goal_r, goal_c = locate(target, val)
                total_dist += abs(r - goal_r) + abs(c - goal_c)
    return total_dist

def generate_neighbors(board):
    neighbors = []
    zero_r, zero_c = locate(board, 0)
    for dr, dc in directions:
        new_r, new_c = zero_r + dr, zero_c + dc
        if 0 <= new_r < 3 and 0 <= new_c < 3:
            new_board = [list(row) for row in board]
            new_board[zero_r][zero_c], new_board[new_r][new_c] = new_board[new_r][new_c], new_board[zero_r][zero_c]
            neighbors.append(new_board)
    return neighbors

def display_board(board):
    for row in board:
        print(' '.join(str(num) for num in row))
    print()

def a_star_search(start, heuristic_func):
    open_set = []
    g_cost = 0

```

```

    f_cost = g_cost + heuristic_func(start)
    heapq.heappush(open_set, (f_cost, g_cost, start, []))    explored
    = set()

    while open_set:
        f_cost, g_cost, current_board, path_so_far = heapq.heappop(open_set)
    if current_board == target:
        return path_so_far + [current_board]

        explored.add(board_to_tuple(current_board))

        for neighbor in generate_neighbors(current_board):
            neighbor_tup = board_to_tuple(neighbor)
        if neighbor_tup not in explored:
            new_g = g_cost + 1
            new_f = new_g + heuristic_func(neighbor)          heapq.heappush(open_set,
            (new_f, new_g, neighbor, path_so_far + [current_board]))

        return None

initial_state = [
    [2, 8, 3],
    [1, 6, 4],
    [7, 0, 5]
]

print("Using Misplaced Tiles Heuristic:") solution_path =
a_star_search(initial_state, misplaced_tiles_heuristic) print("Steps:",
len(solution_path) - 1) for step_num, state in
enumerate(solution_path):
    print(f"Step {step_num}:")
    display_board(state)

print("Using Manhattan Distance Heuristic:")
solution_path = a_star_search(initial_state, manhattan_heuristic)
print("Steps:", len(solution_path) - 1) for
step_num, state in enumerate(solution_path):
    print(f"Step {step_num}:")
    display_board(state)

```

Output:

Using Misplaced Tiles Heuristic:

Steps: 5

Step 0:

2 8 3

1 6 4

7 0 5

Step 1:

2 8 3

1 0 4

7 6 5

Step 2:

2 0 3

1 8 4

7 6 5

Step 3:

0 2 3

1 8 4

7 6 5

Step 4:

1 2 3

0 8 4

7 6 5

Step 5:

1 2 3

8 0 4

7 6 5

Using Manhattan Distance Heuristic:

Steps: 5

Step 0:

2 8 3

1 6 4

7 0 5

Step 1:

2 8 3

1 0 4

7 6 5

Step 2:

2 0 3

1 8 4

7 6 5

Step 3:

0 2 3

1 8 4

7 6 5

Step 4:

1 2 3

0 8 4

7 6 5

Step 5:

1 2 3

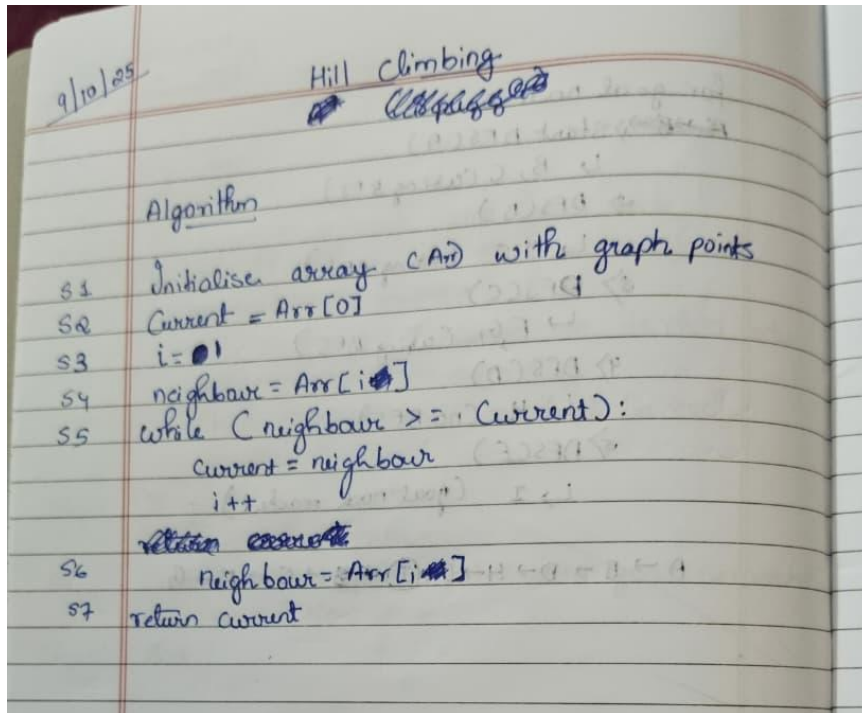
8 0 4

7 6 5

Program 4

Implement Hill Climbing search algorithm to solve N-Queens problem

Algorithm:



Code:

```
import random
```

```
def display(board):
```

```
    n = len(board)    for
```

```
    i in range(n):
```

```
        print(" ".join("Q" if board[i] == j else "." for j in range(n)))
```

```
    print()
```

```
def conflicts(board):    n = len(board)    count = 0    for i in
```

```
    range(n):        for j in range(i + 1, n):            if board[i] == board[j]
```

```
    or abs(board[i] - board[j]) == abs(i - j):
```

```
        count += 1
```

```
    return count
```

```

def best_move(board):    n =
len(board)    best =
list(board)    best_val =
conflicts(board)    for r in
range(n):        for c in
range(n):        if board[r]
!= c:            temp =
list(board)        temp[r]
= c            val =
conflicts(temp)        if
val < best_val:
best_val = val
best = temp    return best,
best_val

def hill_climb(n):    state = [random.randint(0, n -
1) for _ in range(n)]    cost = conflicts(state)
print("Initial Board:")    display(state)
print(f"Initial Cost: {cost}\n")    step = 1    while
True:
    nxt, nxt_cost = best_move(state)
print(f"Step {step}:")    print("Current
Board:")    display(state)
print(f"Current Cost: {cost}")    print(f"Best
Neighbor Cost: {nxt_cost}\n")    if nxt_cost
<= cost:
    break    state, cost =
nxt, nxt_cost    step += 1
print("Final Board:")
display(state)    print(f"Final
Cost: {cost}")    if cost == 0:
    print("Goal State Reached!")
else:
    print("Stuck in Local Minimum!")

hill_climb(4)

```

Output:

```

Initial Board:
. . Q .
. . . Q
. . Q .
. . Q .

Initial Cost: 5

Step 1:
Current Board:
. . Q .
. . . Q
. . Q .
. . Q .

Current Cost: 5
Best Neighbor Cost: 2

Step 2:
Current Board:
. Q . .
. . . Q
. . Q .
. . Q .

Current Cost: 2
Best Neighbor Cost: 0

Step 3:
Current Board:
. Q . .
. . . Q
Q . . .
. . Q .

Current Cost: 0
Best Neighbor Cost: 0

Final Board:
. Q . .
. . . Q
Q . . .
. . Q .

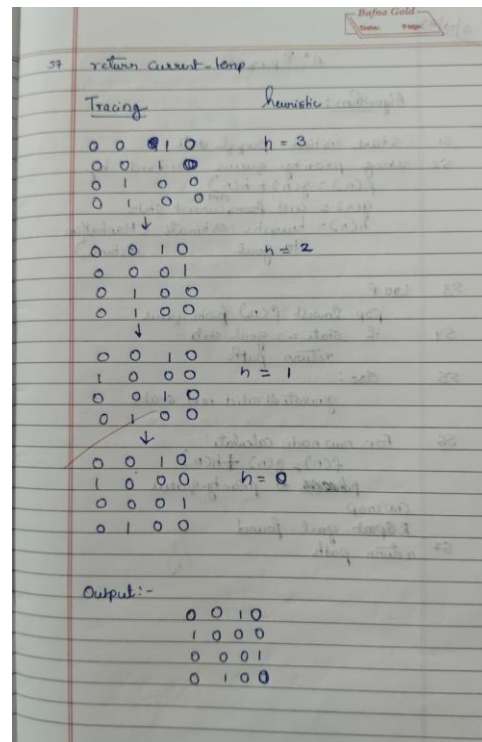
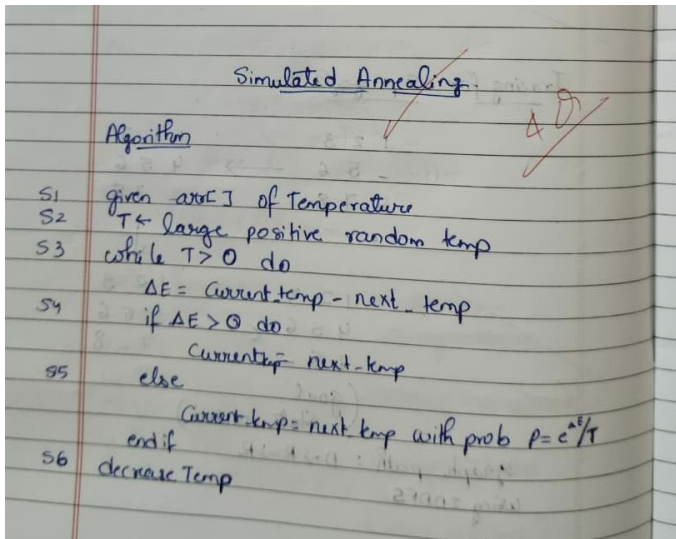
Final Cost: 0
Goal State Reached!

```

Program 5

Simulated Annealing to Solve 8-Queens problem

Algorithm:



Code:

```
import random
import math
```

```
def display(board):
    n = len(board)
    for i in range(n):
        print(" ".join("Q" if board[i] == j else "." for j in range(n)))
    print()
```

```
def cost(board):
    n = len(board)
    c = 0
    for i in range(n):
        for j in range(i + 1, n):
            if board[i] == board[j] or abs(board[i] - board[j]) == abs(i - j):
                c += 1
```

```

    return c

def neighbor(board):
    n = len(board)    b =
list(board)    r =
random.randint(0, n - 1)    c =
random.randint(0, n - 1)
    b[r] = c
    return b

def simulated_annealing(n, t0=100, cool=0.95, t_min=1):
    state = [random.randint(0, n - 1) for _ in range(n)]
    energy = cost(state)    temp = t0    step = 1
    print("Initial Board:")    display(state)    print(f"Initial
Cost: {energy}\n")    while temp > t_min and energy >
0:
        nxt = neighbor(state)    nxt_cost = cost(nxt)    d =
nxt_cost - energy    if d < 0 or random.random() <
math.exp(-d / temp):
            state = nxt    energy =
nxt_cost    print(f"Step {step}: Temp={temp:.3f},
Cost={energy}")    step += 1    temp *= cool
    print("\nFinal Board:")    display(state)    print(f"Final Cost:
{energy}")    if energy == 0:
        print("Goal State Reached!")
    else:
        print("Terminated before reaching goal.")

simulated_annealing(8)

```

Output:

```

Initial Board:
Q . . . . .
. . Q . . .
. . . . . Q
. . . Q . .
. . . Q . .
. . Q . . .
. . . . . Q
. . . Q . .
. . Q . . .
. . . . .

```

Initial Cost: 7

```

Step 1: Temp=100.000, Cost=6
Step 2: Temp=95.000, Cost=6
Step 3: Temp=90.250, Cost=4
Step 4: Temp=85.737, Cost=4
Step 5: Temp=81.451, Cost=5
Step 6: Temp=77.378, Cost=3
Step 7: Temp=73.509, Cost=3
Step 8: Temp=69.834, Cost=6
Step 9: Temp=66.342, Cost=5
Step 10: Temp=63.025, Cost=6
Step 11: Temp=59.874, Cost=6
Step 12: Temp=56.880, Cost=6
Step 13: Temp=54.036, Cost=8
Step 14: Temp=51.334, Cost=10
Step 15: Temp=48.767, Cost=11
Step 16: Temp=46.329, Cost=10
Step 17: Temp=44.013, Cost=11
Step 18: Temp=41.812, Cost=7
Step 19: Temp=39.721, Cost=7
Step 20: Temp=37.735, Cost=4
Step 21: Temp=35.840, Cost=6
Step 22: Temp=34.056, Cost=8
Step 23: Temp=32.353, Cost=8
Step 24: Temp=30.736, Cost=8
Step 25: Temp=29.199, Cost=7
Step 26: Temp=27.739, Cost=7
Step 27: Temp=26.352, Cost=7
Step 28: Temp=25.034, Cost=7

```

```

Step 29: Temp=23.783, Cost=7
Step 30: Temp=22.594, Cost=7
Step 31: Temp=21.464, Cost=5
Step 32: Temp=20.391, Cost=5
Step 33: Temp=19.371, Cost=4
Step 34: Temp=18.403, Cost=8
Step 35: Temp=17.482, Cost=7
Step 36: Temp=16.608, Cost=7
Step 37: Temp=15.778, Cost=7
Step 38: Temp=14.989, Cost=6
Step 39: Temp=14.240, Cost=6
Step 40: Temp=13.528, Cost=6
Step 41: Temp=12.851, Cost=6
Step 42: Temp=12.209, Cost=6
Step 43: Temp=11.598, Cost=5
Step 44: Temp=11.018, Cost=6
Step 45: Temp=10.467, Cost=8
Step 46: Temp=9.944, Cost=8
Step 47: Temp=9.447, Cost=9
Step 48: Temp=8.974, Cost=8
Step 49: Temp=8.526, Cost=10
Step 50: Temp=8.099, Cost=10
Step 51: Temp=7.694, Cost=8
Step 52: Temp=7.310, Cost=8
Step 53: Temp=6.944, Cost=8
Step 54: Temp=6.597, Cost=8
Step 55: Temp=6.267, Cost=8
Step 56: Temp=5.954, Cost=8
Step 57: Temp=5.656, Cost=7
Step 58: Temp=5.373, Cost=5
Step 59: Temp=5.105, Cost=5
Step 60: Temp=4.849, Cost=4
Step 61: Temp=4.607, Cost=6
Step 62: Temp=4.377, Cost=4
Step 63: Temp=4.158, Cost=4
Step 64: Temp=3.950, Cost=5
Step 65: Temp=3.752, Cost=6
Step 66: Temp=3.565, Cost=6
Step 67: Temp=3.387, Cost=5
Step 68: Temp=3.217, Cost=6
Step 69: Temp=3.056, Cost=6

```

```

Step 62: Temp=4.377, Cost=4
Step 63: Temp=4.158, Cost=4
Step 64: Temp=3.950, Cost=5
Step 65: Temp=3.752, Cost=6
Step 66: Temp=3.565, Cost=6
Step 67: Temp=3.387, Cost=5
Step 68: Temp=3.217, Cost=6
Step 69: Temp=3.056, Cost=6
Step 70: Temp=2.904, Cost=6
Step 71: Temp=2.758, Cost=6
Step 72: Temp=2.620, Cost=6
Step 73: Temp=2.489, Cost=6
Step 74: Temp=2.365, Cost=6
Step 75: Temp=2.247, Cost=6
Step 76: Temp=2.134, Cost=6
Step 77: Temp=2.028, Cost=7
Step 78: Temp=1.926, Cost=7
Step 79: Temp=1.830, Cost=7
Step 80: Temp=1.738, Cost=7
Step 81: Temp=1.652, Cost=6
Step 82: Temp=1.569, Cost=6
Step 83: Temp=1.491, Cost=6
Step 84: Temp=1.416, Cost=5
Step 85: Temp=1.345, Cost=6
Step 86: Temp=1.278, Cost=6
Step 87: Temp=1.214, Cost=7
Step 88: Temp=1.153, Cost=8
Step 89: Temp=1.096, Cost=8
Step 90: Temp=1.041, Cost=8

```

Final Board:

```

. . . . Q . .
. . . . Q . .
. . . Q . . .
. . . Q . . .
. . . Q . . .
Q . . . . .
. . Q . . .
. . . . . Q

```

Final Cost: 8
Terminated before reaching goal.

Program 6

Create a knowledge base using propositional logic and show that the given query entails the knowledge base or not.

Algorithm:

14/10/2020

Knowledge Base (Propositional Logic)

PseudoCode (Algorithm)

Function TT_check - Enumeration: It returns True or False
input: $KB \leftarrow$ knowledge base, a propositional logic
 $\alpha \leftarrow$ query symbol
Symbols $\leftarrow$ list of propositional logic symbols
return TT_check All (KB, α , Symbols)

Function TT_check All returns True or False
if Empty? (Symbols) True
if True then return True? (α , model)
else return True
else do
 $P \leftarrow$ First (Symbols)
 $\alpha \leftarrow$ Rest (Symbols)
return TT_check All (KB, α , P)
and TT_check All (KB, α , P) ? $P = \text{True}$?
Output:-
True $\rightarrow$ if KB $\models \alpha$
false $\rightarrow$ otherwise

17

Truth Table

P	Q	R	$\neg Q$	$Q \rightarrow P$	$P \rightarrow \neg Q$	$Q \vee R$	KB
0	0	0	1	1	1	0	0
0	0	1	1	1	1	1	1
0	1	0	0	0	1	1	0
0	1	1	0	0	1	1	0
1	0	0	1	1	1	0	0
1	0	1	1	1	1	1	1
1	1	0	0	0	0	1	0
1	1	1	0	0	0	1	0

ii) KB entails R ?

KB	R
1	1
1	1
1	1

Therefore KB entails R is true

iii) KB entails $R \rightarrow P$?

KB	$R \rightarrow P$
1	0
1	1

KB entails $R \rightarrow P$ is false

ii) KB entails $Q \rightarrow R$

KB	$Q \rightarrow R$
1	1
1	1

KB entails $Q \rightarrow R$ is true

Output: $\alpha = A \vee B$ KB = $(A \vee C) \wedge (C \vee \neg C)$

Full Truth Table:

A	B	C	α	KB
0	0	0	0	0
0	0	1	0	0
0	1	0	1	0
0	1	1	1	1
1	0	0	1	1
1	0	1	1	0
1	1	0	1	1
1	1	1	1	1

Models where KB is True

A	B	C	α	KB
0	1	1	1	1
1	0	0	1	1
1	1	0	1	1
1	1	1	1	1

Does KB entail $\alpha (A \vee B)$? yes

Code:

```
import itertools
import re
```

```
def eval_expr(expr, model):
    expr = expr.replace("<->", "==")
    expr = expr.replace(">", "<=")
    expr = re.sub(r'~(\w+)', r'(not \1)', expr)
    expr = re.sub(r'~(((\^)+))', r'(not (\1))', expr)
    expr = expr.replace("^", "and")
    expr = expr.replace("v", "or")
    for s, v in model.items():
        expr = re.sub(r'\b' + re.escape(s) + r'\b', str(v), expr)
    return eval(expr)
```



```

def truth_table_entails(kb, query, symbols):
    entail = True    combos = list(itertools.product([True, False],
repeat=len(symbols)))    print("Truth Table Evaluation:\n")
    head = " | ".join(symbols) + " | KB | Query | KB  $\Rightarrow$  Query"
    print(head)    print("-" * len(head) * 2)    for vals in
combos:
        model = dict(zip(symbols, vals))
        kb_val = eval_expr(kb, model)
        q_val = eval_expr(query, model)
        implies = (not kb_val) or q_val        if
kb_val and not q_val:
            entail = False
            row = " | ".join(['T' if v else 'F' for v in vals])        row += f' | {'T' if kb_val else
'F'} | {'T' if q_val else 'F'} | {'T' if implies else 'F'}"        print(row)
    print("\nResult:")    if entail:
        print("The Knowledge Base entails the Query (KB  $\models$  Query)")
    else:
        print("The Knowledge Base does NOT entail the Query (KB  $\not\models$  Query)")

kb = "(Q -> P) ^ (P -> ~Q) ^ (Q v R)" symbols
= ["P", "Q", "R"]
queries = ["R", "R -> P", "Q -> R"]

for q in queries:
    print(f"\nEvaluating Query: {q}\n")
    truth_table_entails(kb, q, symbols)
    print("\n" + "="*50 + "\n")

```

Output:

```

P | Q | R | KB | Query | KB  $\Rightarrow$  Query
-----
T | T | T | F | T | T
T | T | F | F | F | T
T | F | T | T | T | T
T | F | F | F | F | T
F | T | T | F | T | T
F | T | F | F | F | T
F | F | T | T | T | T
F | F | F | F | F | T
F | F | F | F | F | T

Result:
The Knowledge Base entails the Query (KB  $\models$  Query)

=====

Evaluating Query: R -> P
Truth Table Evaluation:
P | Q | R | KB | Query | KB  $\Rightarrow$  Query
-----
T | T | T | F | T | T
T | T | F | F | F | T
T | F | T | T | T | T
T | F | F | F | F | T
F | T | T | F | T | T
F | T | F | F | F | T
F | F | T | T | T | T
F | F | F | F | F | T
F | F | F | F | F | T

Result:
The Knowledge Base does NOT entail the Query (KB  $\not\models$  Query)

=====

Evaluating Query: Q -> R
Truth Table Evaluation:
P | Q | R | KB | Query | KB  $\Rightarrow$  Query
-----
T | T | T | F | T | T
T | T | F | F | F | T
T | F | T | T | T | T
T | F | F | F | F | T
F | T | T | F | T | T
F | T | F | F | F | T
F | F | T | T | T | T
F | F | F | F | F | T
F | F | F | F | F | T

```

```

Result:
The Knowledge Base does NOT entail the Query (KB  $\not\models$  Query)

=====

Evaluating Query: Q -> R
Truth Table Evaluation:
P | Q | R | KB | Query | KB  $\Rightarrow$  Query
-----
T | T | T | F | T | T
T | T | F | F | F | T
T | F | T | T | T | T
T | F | F | F | F | T
F | T | T | F | T | T
F | T | F | F | F | T
F | F | T | T | T | T
F | F | F | F | F | T
F | F | F | F | F | T

Result:
The Knowledge Base entails the Query (KB  $\models$  Query)

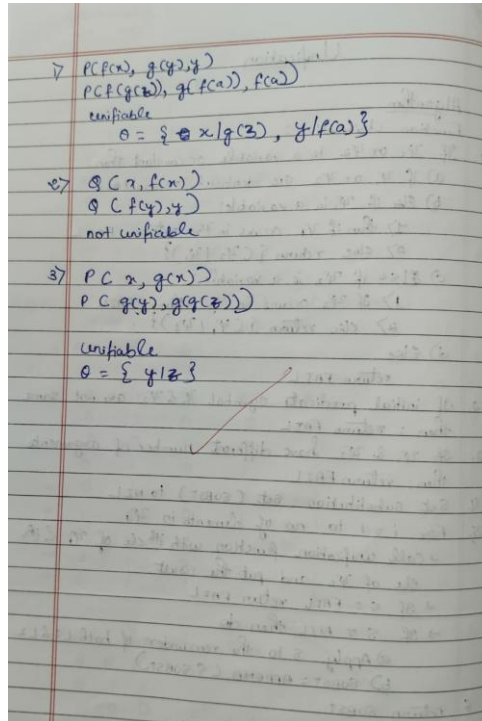
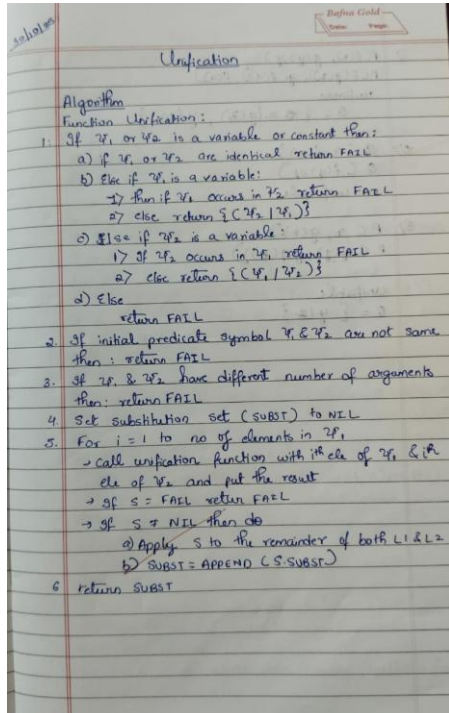
=====

```

Program 7

Implement unification in first order logic

Algorithm:



Code:

def var(x):

return isinstance(x, str) and x.islower()

def const(x):

return isinstance(x, str) and x[0].isupper()

def occurs(v, e, s): if

v == e: return True

elif isinstance(e, list):

return any(occurs(v, el, s) for el in e)

elif e in s: return occurs(v, s[e], s)

return False

```

def unify(a, b, s=None, d=0):
    tab = " " * d    if s is None:
print(tab + "Substitution failed.")
return None
    print(tab + f"Unify({a}, {b}) with subst = {s}")
if a == b:    print(tab + "Terms are identical, no
change.")    return s    elif var(a):
    return unify_var(a, b, s, d)
elif var(b):
    return unify_var(b, a, s, d)    elif isinstance(a,
list) and isinstance(b, list):    if len(a) != len(b):
print(tab + "Lists have different lengths. Fail.")
return None    for x1, x2 in zip(a, b):
    s = unify(x1, x2, s, d + 1)    if s is
None:    print(tab + "Failed to unify list
elements.")
    return None
return s    else:
    print(tab + "Cannot unify different constants or structures. Fail.")
return None

```

```

def unify_var(v, x, s, d):
    tab = " " * d
if v in s:
    print(tab + f"{v} is in subst, unify({s[v]}, {x})")
return unify(s[v], x, s, d + 1)    elif var(x) and x in s:
    print(tab + f"{x} is in subst, unify({v}, {s[x]})")
return unify(v, s[x], s, d + 1)    elif occurs(v, x, s):
print(tab + f"Occurs check failed: {v} occurs in {x}")
return None    else:
    print(tab + f"Add {v} -> {x} to subst")
    s[v] = x
    return s

```

```

e1 = ['f', 'X', ['g', 'Y']] e2
= ['f', 'a', ['g', 'b']]

```

```

print("Starting Unification:\n") res
= unify(e1, e2, s={})
print("\nFinal Unification Result:", res)

```

Output:



Starting Unification:

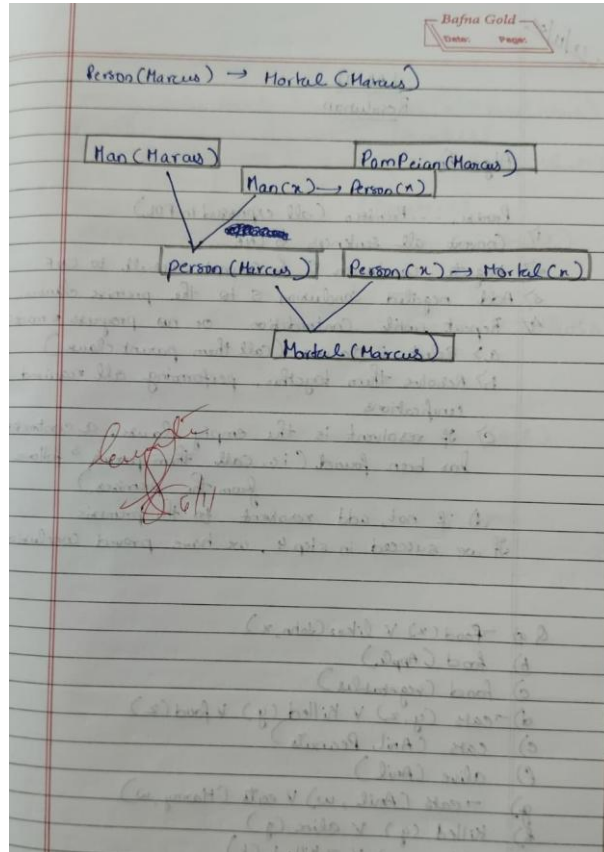
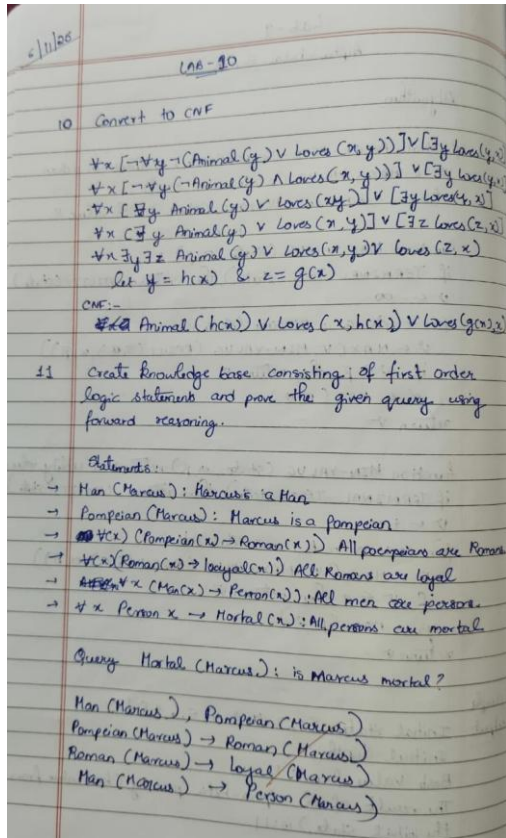
```
Unify(['f', 'X', ['g', 'Y']], ['f', 'a', ['g', 'b']]) with subst = {}  
  Unify(f, f) with subst = {}  
    Terms are identical, no change.  
  Unify(X, a) with subst = {}  
    Add a -> X to subst  
  Unify(['g', 'Y'], ['g', 'b']) with subst = {'a': 'X'}  
    Unify(g, g) with subst = {'a': 'X'}  
      Terms are identical, no change.  
    Unify(Y, b) with subst = {'a': 'X'}  
      Add b -> Y to subst
```

Final Unification Result: {'a': 'X', 'b': 'Y'}

Program 8

Create a knowledge base consisting of first order logic statements and prove the given query using forward reasoning.

Algorithm:



Code:

```
# -*- coding: utf-8 -*-
from copy import deepcopy
import networkx as nx
import matplotlib.pyplot as plt
```

```
KB = [
    {"if": ["American(p)", "Weapon(q)", "Sells(p, q, r)", "Hostile(r)"], "then": "Criminal(p)"},
    {"if": ["Missile(x)", "then": "Weapon(x)"},
    {"if": ["Enemy(x, America)", "then": "Hostile(x)"},
    {"if": ["Missile(x)", "Owns(A, x)", "then": "Sells(Robert, x, A)"},
    {"fact": "American(Robert)"},
    {"fact": "Enemy(A, America)"},
    {"fact": "Missile(T1)"},
```

```

    {"fact": "Owns(A, T1)"}
]

```

```

goal = "Criminal(Robert)"
def parse_predicate(expr):    pred, args_str
= expr.split("(")    args = args_str[:-
1].split(",")    return pred.strip(), [a.strip()
for a in args]

```

```

def is_variable(term):
return term[0].islower()

```

```

def unify(expr1, expr2, subs=None):
"""Unify two predicates expr1 and expr2."""
if subs is None:    subs = {}
    p1, args1 = parse_predicate(expr1)
    p2, args2 = parse_predicate(expr2)    if
p1 != p2 or len(args1) != len(args2):
        return None    for t1, t2 in
zip(args1, args2):    if t1 ==
t2:    continue    elif
is_variable(t1):    subs[t1]
= t2    elif is_variable(t2):
subs[t2] = t1    else:
        return None
    return subs

```

```

def substitute(expr, subs):
    pred, args = parse_predicate(expr)
    new_args = []    for a in args:
while a in subs:    a = subs[a]
    new_args.append(a)
    return f'{pred}({', '.join(new_args)})'

```

```

def FOL_FC_ASK(KB, query):
    known_facts = {item["fact"] for item in KB if "fact" in item}
    rules = [item for item in KB if "if" in item]    inference_edges
= []

```

```

    print("Initial known facts:")
    for f in known_facts:
        print(" ", f)
    print()

    new_facts_added = True

    while new_facts_added:
        new_facts_added = False

        for rule in rules:
            premises = rule["if"]
            conclusion = rule["then"]

            substitutions_list = [{}]

            for premise in premises:
                new_substitutions = []
                for subs in substitutions_list:
                    premise_substituted = substitute(premise, subs)
                    for fact in known_facts:
                        new_subs = unify(premise_substituted, fact, deepcopy(subs))
                    if new_subs is not None:
                        new_substitutions.append(new_subs)
                substitutions_list = new_substitutions
            if not substitutions_list:
                break

            for subs in substitutions_list:
                inferred_fact = substitute(conclusion, subs)
                if inferred_fact not in known_facts:
                    print(f"Inferred: {inferred_fact}")
                    known_facts.add(inferred_fact)
                    new_facts_added = True

            # record edges for graph
            for premise in premises:
                inference_edges.append((substitute(premise, subs), inferred_fact))

            # stop if query is reached
            if unify(inferred_fact, query):
                print("\nQuery satisfied:", query)
                visualize_graph(inference_edges, query)
                return True

```

```

    print("\nQuery cannot be proved.")
visualize_graph(inference_edges, query)    return
False

def visualize_graph(edges, goal):
    G = nx.DiGraph()
    G.add_edges_from(edges)

    plt.figure(figsize=(12, 7))    pos =
    nx.spring_layout(G, seed=42)

    node_colors = ["lightgreen" if n == goal else "lightblue" for n in G.nodes()]
    nx.draw(
        G, pos, with_labels=True, node_color=node_colors,
        node_size=2200, font_size=10, font_weight="bold",    arrows=True,
        arrowstyle="->", arrowsize=12
    )

    plt.title("Forward Chaining Inference Graph", fontsize=14, fontweight="bold")
    plt.show()

print("\n--- Forward Chaining (FOL-FC-ASK) ---\n")
FOL_FC_ASK(KB, goal)

```

Output:

Code:

```
FOL_data = {
    'a': "∀x: food(x) → likes(John, x)",
    'b': "food(Apple) ∧ food(Vegetables)",
    'c': "∀x∀y: eats(x, y) ∧ ¬killed(x) → food(y)",
    'd': "eats(Anil, Peanuts) ∧ alive(Anil)",
    'e': "∀x: eats(Anil, x) → eats(Harry, x)",
    'f': "∀x: ¬killed(x) → alive(x)",
    'g': "∀x: alive(x) → ¬killed(x)",
    'h': "likes(John, Peanuts)"
}

print("=== STEP 1: Given FOL Statements ===") for
label, formula in FOL_data.items():
    print(f'{label}. {formula}')

print("\n=== STEP 2: After Removing Implications ===")

CNF_no_implications = {
    'a': "¬food(x) ∨ likes(John, x)",
    'b1': "food(Apple)",
    'b2': "food(Vegetables)",
    'c': "¬eats(x, y) ∨ killed(x) ∨ food(y)",
    'd1': "eats(Anil, Peanuts)",
    'd2': "alive(Anil)",
    'e': "¬eats(Anil, x) ∨ eats(Harry, x)",
    'f': "killed(x) ∨ alive(x)",
    'g': "¬alive(x) ∨ ¬killed(x)",
    'h': "likes(John, Peanuts)"
}

for label, clause in CNF_no_implications.items():
    print(f'{label}. {clause}')

print("\n=== STEP 3: Standardized Variables (Dropped Quantifiers) ===") for
key, val in CNF_no_implications.items():
    print(f'{key}. {val}')
```

```
print("\n=== STEP 4: Final CNF Clauses ===")
```

```
CNF_final = [  
    "¬food(x) ∨ likes(John, x)",  
    "food(Apple)",  
    "food(Vegetables)",  
    "¬eats(y, z) ∨ killed(y) ∨ food(z)",  
    "eats(Anil, Peanuts)",  
    "alive(Anil)",  
    "¬eats(Anil, w) ∨ eats(Harry, w)",  
    "killed(g) ∨ alive(g)",  
    "¬alive(k) ∨ ¬killed(k)",  
    "likes(John, Peanuts)"  
]
```

```
for i, clause in enumerate(CNF_final, start=1):  
    print(f'{i}. {clause}')
```

```
print("\n=== STEP 5: Resolution Proof ===")
```

```
resolution_steps = [  
    ("1", "Negate the Goal", "¬likes(John, Peanuts)"),  
    ("2", "Resolve (1) with (¬food(x) ∨ likes(John, x)) using {x/Peanuts}", "¬food(Peanuts)"),  
    ("3", "Resolve (2) with (¬eats(y,z) ∨ killed(y) ∨ food(z)) using {z/Peanuts}", "¬eats(y,Peanuts) ∨  
killed(y)"),  
    ("4", "Resolve (3) with (eats(Anil, Peanuts)) using {y/Anil}", "killed(Anil)"),  
    ("5", "Resolve (4) with (¬alive(k) ∨ ¬killed(k)) using {k/Anil}", "¬alive(Anil)"),  
    ("6", "Resolve (5) with (alive(Anil))", "⊥ (Contradiction)")  
]
```

```
for step_num, description, outcome in resolution_steps:
```

```
    print(f'Step {step_num}: {description}')  
    print(f'    ⇒ {outcome}\n')
```

```
print("Contradiction reached ⇒ Therefore, John likes Peanuts is TRUE.\n")
```

Output:

```

=== STEP 1: Given FOL Statements ===
a.  $\forall x: \text{food}(x) \rightarrow \text{likes}(\text{John}, x)$ 
b.  $\text{food}(\text{Apple}) \wedge \text{food}(\text{Vegetables})$ 
c.  $\forall x \forall y: \text{eats}(x, y) \wedge \neg \text{killed}(x) \rightarrow \text{food}(y)$ 
d.  $\text{eats}(\text{Anil}, \text{Peanuts}) \wedge \text{alive}(\text{Anil})$ 
e.  $\forall x: \text{eats}(\text{Anil}, x) \rightarrow \text{eats}(\text{Harry}, x)$ 
f.  $\forall x: \neg \text{killed}(x) \rightarrow \text{alive}(x)$ 
g.  $\forall x: \text{alive}(x) \rightarrow \neg \text{killed}(x)$ 
h.  $\text{likes}(\text{John}, \text{Peanuts})$ 

=== STEP 2: After Removing Implications ===
a.  $\neg \text{food}(x) \vee \text{likes}(\text{John}, x)$ 
b1.  $\text{food}(\text{Apple})$ 
b2.  $\text{food}(\text{Vegetables})$ 
c.  $\neg \text{eats}(x, y) \vee \text{killed}(x) \vee \text{food}(y)$ 
d1.  $\text{eats}(\text{Anil}, \text{Peanuts})$ 
d2.  $\text{alive}(\text{Anil})$ 
e.  $\neg \text{eats}(\text{Anil}, x) \vee \text{eats}(\text{Harry}, x)$ 
f.  $\text{killed}(x) \vee \text{alive}(x)$ 
g.  $\neg \text{alive}(x) \vee \neg \text{killed}(x)$ 
h.  $\text{likes}(\text{John}, \text{Peanuts})$ 

=== STEP 3: Standardized Variables (Dropped Quantifiers) ===
a.  $\neg \text{food}(x) \vee \text{likes}(\text{John}, x)$ 
b1.  $\text{food}(\text{Apple})$ 
b2.  $\text{food}(\text{Vegetables})$ 
c.  $\neg \text{eats}(x, y) \vee \text{killed}(x) \vee \text{food}(y)$ 
d1.  $\text{eats}(\text{Anil}, \text{Peanuts})$ 
d2.  $\text{alive}(\text{Anil})$ 
e.  $\neg \text{eats}(\text{Anil}, x) \vee \text{eats}(\text{Harry}, x)$ 
f.  $\text{killed}(x) \vee \text{alive}(x)$ 
g.  $\neg \text{alive}(x) \vee \neg \text{killed}(x)$ 
h.  $\text{likes}(\text{John}, \text{Peanuts})$ 

=== STEP 4: Final CNF Clauses ===
1.  $\neg \text{food}(x) \vee \text{likes}(\text{John}, x)$ 
2.  $\text{food}(\text{Apple})$ 
3.  $\text{food}(\text{Vegetables})$ 
4.  $\neg \text{eats}(y, z) \vee \text{killed}(y) \vee \text{food}(z)$ 
5.  $\text{eats}(\text{Anil}, \text{Peanuts})$ 
6.  $\text{alive}(\text{Anil})$ 
7.  $\neg \text{eats}(\text{Anil}, u) \vee \text{eats}(\text{Harry}, u)$ 
8.  $\text{killed}(g) \vee \text{alive}(g)$ 
9.  $\neg \text{alive}(k) \vee \neg \text{killed}(k)$ 
10.  $\text{likes}(\text{John}, \text{Peanuts})$ 

```

=== STEP 5: Resolution Proof ===

Step 1: Negate the Goal

$\Rightarrow \neg \text{likes}(\text{John}, \text{Peanuts})$

Step 2: Resolve (1) with $(\neg \text{food}(x) \vee \text{likes}(\text{John}, x))$ using $\{x/\text{Peanuts}\}$

$\Rightarrow \neg \text{food}(\text{Peanuts})$

Step 3: Resolve (2) with $(\neg \text{eats}(y, z) \vee \text{killed}(y) \vee \text{food}(z))$ using $\{z/\text{Peanuts}\}$

$\Rightarrow \neg \text{eats}(y, \text{Peanuts}) \vee \text{killed}(y)$

Step 4: Resolve (3) with $(\text{eats}(\text{Anil}, \text{Peanuts}))$ using $\{y/\text{Anil}\}$

$\Rightarrow \text{killed}(\text{Anil})$

Step 5: Resolve (4) with $(\neg \text{alive}(k) \vee \neg \text{killed}(k))$ using $\{k/\text{Anil}\}$

$\Rightarrow \neg \text{alive}(\text{Anil})$

Step 6: Resolve (5) with $(\text{alive}(\text{Anil}))$

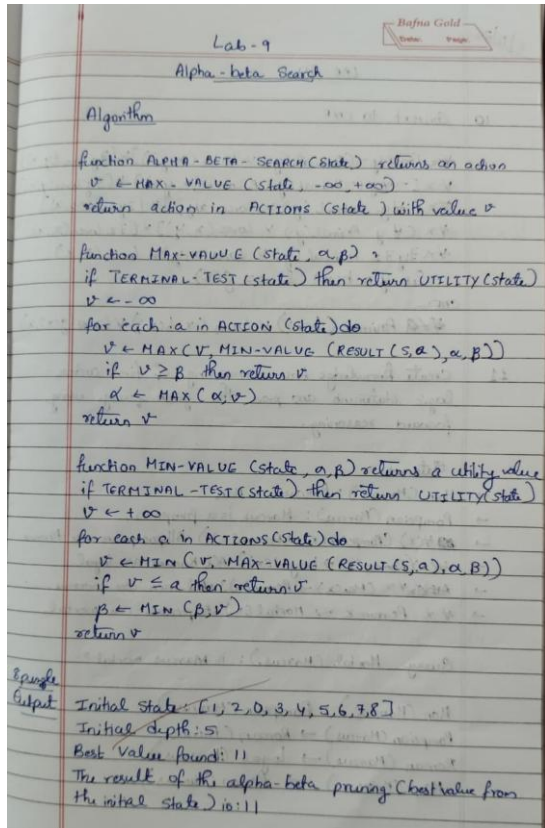
$\Rightarrow \perp$ (Contradiction)

Contradiction reached $\Rightarrow$ Therefore, John likes Peanuts is TRUE.

Program 10

Implement Alpha-Beta Pruning.

Algorithm:



Code:

```
import math
```

```
tree = {  
    'A': ['B', 'C'],  
    'B': ['D', 'E'],  
    'C': ['F', 'G'],  
    'D': ['L1', 'L2'],  
    'E': ['L3', 'L4'],  
    'F': ['L5', 'L6'],  
    'G': ['L7', 'L8'],  
    'L1': 10,  
    'L2': 9,  
    'L3': 14,  
    'L4': 18,  
    'L5': 5,  
    'L6': 4,
```

```

'L7': 50,
'L8': 3
}

```

```

def show_tree():
    print("\nGame Tree Structure:\n")
    print("          A (MAX)")
    print("        /   \\\")  print("
B (MIN)    C (MIN)")
    print("      /  \\\  /  \\\")
    print("    D (MAX) E (MAX) F (MAX) G (MAX)")
    print("  /  \\\  /  \\\  /  \\\  /  \\\")  print("
10  9 14 18  5  4  50  3")  print("\n-----
-----\n")

```

```

def alpha_beta(node, depth, alpha, beta, is_max):
    pad = " " * depth

```

```

    if isinstance(tree[node], int):
        print(f'{pad}Reached leaf {node} with value {tree[node]}')
    return tree[node]

```

```

    if is_max:
        print(f'{pad}Exploring MAX node {node} (depth={depth}),  $\alpha$ = {alpha},  $\beta$ = {beta}')
        value = -math.inf
        for child in tree[node]:
            print(f'{pad}--> Exploring child {child} of {node}')
            score = alpha_beta(child, depth + 1, alpha, beta, False)
            value = max(value, score)
            alpha = max(alpha, score)
            print(f'{pad}Updated MAX node {node}: value={value},  $\alpha$ = {alpha},  $\beta$ = {beta}')
            if beta <= alpha:
                print(f'{pad}!!! Pruning at MAX node {node} ( $\beta$ = {beta}  $\leq$   $\alpha$ = {alpha})')
                break
        return value

```

```

    else:
        print(f'{pad}Exploring MIN node {node} (depth={depth}),  $\alpha$ = {alpha},  $\beta$ = {beta}')
        value = math.inf
        for child in tree[node]:
            print(f'{pad}--> Exploring child {child} of {node}')
            score = alpha_beta(child, depth + 1, alpha, beta, True)
            value = min(value, score)
            beta = min(beta, score)
            print(f'{pad}Updated MIN node {node}: value={value},  $\alpha$ = {alpha},  $\beta$ = {beta}')
            if beta <= alpha:

```

```

        print(f' {pad}!!! Pruning at MIN node {node} ( $\beta=\{\text{beta}\} \leq \alpha=\{\text{alpha}\}$ )')
break
    return value

```

```

show_tree() print("Starting Alpha-Beta
Pruning...\n")

```

```

result = alpha_beta('A', 0, -math.inf, math.inf, True)

```

```

print("\n-----") print(f'✅
Best achievable value at root (A): {result}')
print("-----")

```

Output:

```

Game Tree Structure:
  A (MAX)
 /  \
B (MIN) C (MIN)
 /  \  /  \
D (MAX) E (MAX) F (MAX) G (MAX)
/  \  /  \  /  \
10 9 14 18 5 4 50 3

-----

Starting Alpha-Beta Pruning...

Exploring MAX node A (depth=0),  $\alpha=-\text{inf}$ ,  $\beta=-\text{inf}$ 
--> Exploring child B of A
  Exploring MIN node B (depth=1),  $\alpha=-\text{inf}$ ,  $\beta=-\text{inf}$ 
  --> Exploring child D of B
    Exploring MAX node D (depth=2),  $\alpha=-\text{inf}$ ,  $\beta=-\text{inf}$ 
    --> Exploring child L1 of D
      Reached leaf L1 with value 10
      Updated MAX node D: value=10,  $\alpha=10$ ,  $\beta=-\text{inf}$ 
    --> Exploring child L2 of D
      Reached leaf L2 with value 9
      Updated MAX node D: value=10,  $\alpha=10$ ,  $\beta=-\text{inf}$ 
    Updated MIN node B: value=10,  $\alpha=-\text{inf}$ ,  $\beta=10$ 
  --> Exploring child E of B
    Exploring MAX node E (depth=2),  $\alpha=-\text{inf}$ ,  $\beta=10$ 
    --> Exploring child L3 of E
      Reached leaf L3 with value 14
      Updated MAX node E: value=14,  $\alpha=14$ ,  $\beta=10$ 
    !!! Pruning at MAX node E ( $\beta=10 \leq \alpha=14$ )
    Updated MIN node B: value=10,  $\alpha=-\text{inf}$ ,  $\beta=10$ 
  Updated MAX node A: value=10,  $\alpha=10$ ,  $\beta=-\text{inf}$ 
--> Exploring child C of A
  Exploring MIN node C (depth=1),  $\alpha=10$ ,  $\beta=-\text{inf}$ 
  --> Exploring child F of C
    Exploring MAX node F (depth=2),  $\alpha=10$ ,  $\beta=-\text{inf}$ 
    --> Exploring child L5 of F
      Reached leaf L5 with value 5
      Updated MAX node F: value=5,  $\alpha=5$ ,  $\beta=-\text{inf}$ 
    --> Exploring child L6 of F
      Reached leaf L6 with value 4
      Updated MAX node F: value=5,  $\alpha=5$ ,  $\beta=-\text{inf}$ 
    Updated MIN node C: value=5,  $\alpha=10$ ,  $\beta=5$ 
  !!! Pruning at MIN node C ( $\beta=5 \leq \alpha=10$ )
  Updated MAX node A: value=10,  $\alpha=10$ ,  $\beta=-\text{inf}$ 

```

```

-----
✅ Best achievable value at root (A): 10
-----

```